Improving Heuristics for A* Search in the Traveling Salesman Problem

Tusshar Gopaul

University of New Brunswick
h9g3p@unb.ca

Abstract. This paper studies heuristic design for solving the Traveling Salesman Problem (TSP) using A* search. A baseline heuristic is compared against an MST-based heuristic and a multi-start nearest neighbor approach. The results show that stronger heuristics can significantly reduce node expansions and runtime, although the TSP remains computationally expensive as the number of cities increases. The study highlights both the strengths and limitations of heuristic search on combinatorial optimization problems.

Keywords: Artificial Intelligence · A* Search · Traveling Salesman Problem · Heuristics

1 Introduction

The Traveling Salesman Problem (TSP) is a classic optimization problem in artificial intelligence and computer science. The objective is to find the shortest possible route that visits each city exactly once and returns to the starting city. I chose this topic because it connects directly to the course material on search, heuristics, and problem-solving agents. Through this project, I learned how strongly heuristic quality affects the efficiency of A* search.

This paper compares three approaches: a simple baseline heuristic, an MST-based heuristic, and a multi-start nearest neighbor method. The main goal is to evaluate how research-inspired heuristics affect runtime, node expansions, and solution quality.

2 The Problem

Imagine a delivery driver who must visit several cities exactly once and then return home. The driver wants the total trip to be as short as possible. That is the Traveling Salesman Problem.

The challenge is that the number of possible tours grows very quickly as the number of cities increases. Because of this, checking every possible route becomes impractical even for moderately sized datasets.

3 Key Idea

The main idea of this project is to improve the heuristic guidance used by A* search.

A* evaluates each search state using:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual cost so far and $h(n)$ is the estimated remaining cost.

If $h(n)$ is too weak, A* explores many unnecessary states. If $h(n)$ is stronger while still admissible, A* can focus on more promising partial tours.

4 Details

4.1 Problem Formulation

A state is represented as a partial tour. An action adds one unvisited city to the tour. A goal state is reached when all cities have been visited and the path returns to the start city. The path cost is the total Euclidean distance traveled.

4.2 Baseline Heuristic

The baseline heuristic provides a simple lower bound on the remaining cost by estimating the minimum possible distance required to complete the tour. It does this by identifying the shortest available edge among the remaining cities and multiplying it by the number of remaining steps.

Formally, the heuristic assumes that each remaining move will cost at least the length of the shortest edge in the graph. While this guarantees admissibility (i.e., the heuristic never overestimates the true cost), it does not capture the structure of the remaining subproblem.

The main advantage of this heuristic is its computational efficiency. It is extremely fast to compute and adds very little overhead to each node expansion in A* search.

However, its major limitation is that it provides a very weak estimate of the remaining cost. Since it ignores how cities are arranged spatially, it often underestimates the true cost significantly. As a result, A* behaves closer to uninformed search, expanding a large number of unnecessary states.

This makes the baseline heuristic useful primarily as a reference point for evaluating improvements in heuristic design rather than as a competitive approach on its own.

5 Results: Baseline Heuristic

This section presents the experimental results obtained using the baseline heuristic in A* search. The goal is to analyze its performance in terms of solution quality, runtime, and search efficiency.

5.1 Single Run Output

```
=== Baseline Heuristic ===
Path: [0, 2, 1, 3, 7, 4, 5, 6, 0]
Tour cost: 37.83
Nodes expanded: 505
Elapsed time (ms): 4.93

=== Improved Heuristic (MST-based) ===
Path: [0, 2, 1, 3, 7, 4, 5, 6, 0]
Tour cost: 37.83
Nodes expanded: 65
Elapsed time (ms): 1.6
```

Fig. 1. Baseline heuristic console output (example run)

The baseline heuristic produced a valid tour that visits all cities and returns to the starting point. As shown in Figure 1, the resulting path was:

- Path: [0, 2, 1, 3, 7, 4, 5, 6, 0]
- Tour cost: 37.83
- Nodes expanded: 505
- Runtime: 4.93 ms

Although the solution cost is optimal, the number of nodes expanded is relatively high. This indicates that the baseline heuristic provides weak guidance to the A* algorithm, causing it to explore many unnecessary states.

5.2 Runtime Comparison (8 Cities)

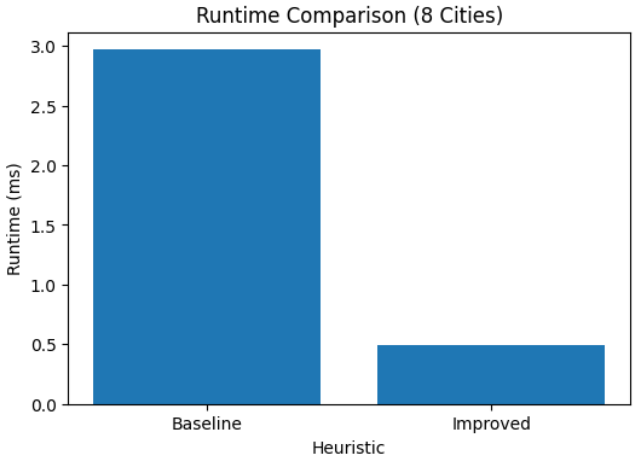

Fig. 2. Runtime comparison for 8-city dataset

Figure 2 shows the runtime of the baseline heuristic for an 8-city dataset. The baseline approach required approximately 3 milliseconds to complete the search.

While this runtime is still relatively low, it is noticeably higher than more informed heuristics. This is due to the large number of nodes explored during the search process.

5.3 Nodes Expanded (8 Cities)

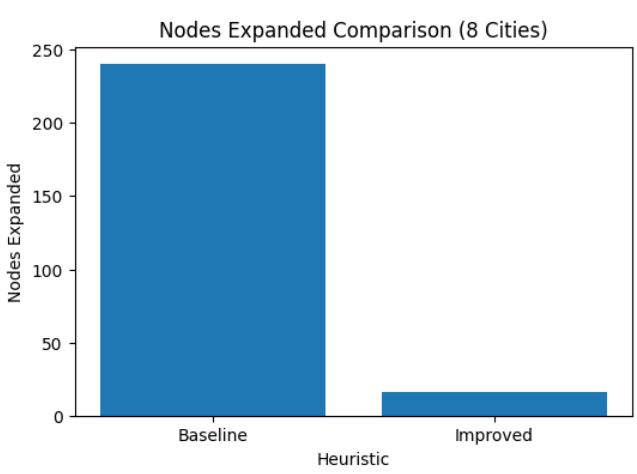

Fig. 3. Nodes expanded for 8-city dataset

As illustrated in Figure 3, the baseline heuristic expanded a significantly large number of nodes even for a small dataset.

This confirms that the heuristic does not effectively guide the search toward promising solutions. Instead, A* behaves closer to uninformed search, exploring many possible partial tours.

5.4 Performance Across Dataset Sizes

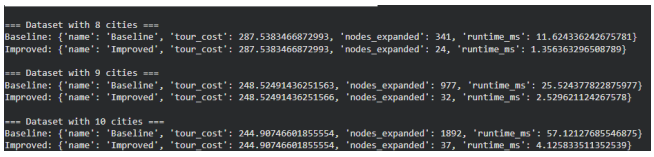

Fig. 4. Nodes expanded across dataset sizes

Figure 4 shows how the number of nodes expanded increases as the number of cities grows.

The results demonstrate a rapid growth pattern:

– 8 cities: 341 nodes expanded

– 9 cities: 977 nodes expanded
– 10 cities: 1892 nodes expanded

This exponential increase highlights the poor scalability of the baseline heuristic.

5.5 Runtime Growth Across Dataset Sizes

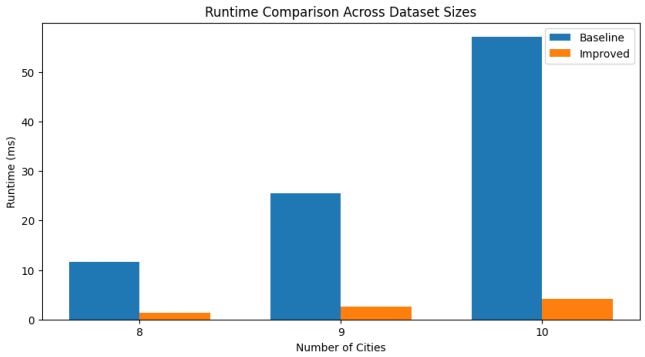

Fig. 5. Runtime across dataset sizes

Figure 5 illustrates the runtime growth as the dataset size increases.

The runtime increases significantly:

– 8 cities: 11.62 ms
– 9 cities: 25.52 ms
– 10 cities: 57.12 ms

This growth is directly correlated with the increase in node expansions, confirming that the baseline heuristic becomes inefficient as problem size increases.

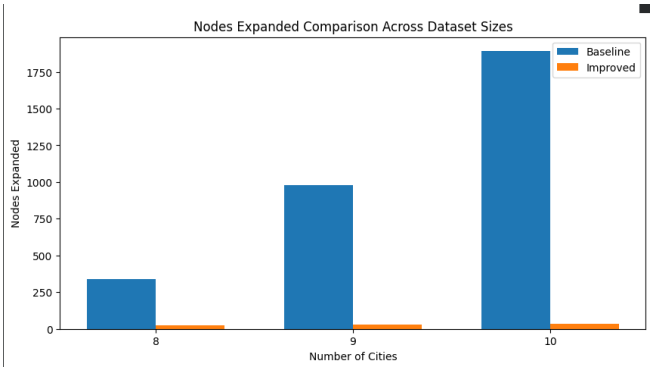

Fig. 6. Nodes expanded across dataset sizes

Figure 6 illustrates the nodes expanded as the dataset size increases, across datasets.

5.6 Summary of Baseline Performance

Overall, the baseline heuristic is effective in maintaining admissibility and producing optimal solutions. However, it suffers from several important limitations:

- It expands a large number of nodes due to weak cost estimation.
- It does not scale well as the number of cities increases.
- It provides limited guidance to the A* search process.

These results highlight the need for stronger heuristics that incorporate more structural information about the problem, motivating the use of the MST-based heuristic explored in the next section.

5.7 MST-Based Heuristic

The improved heuristic enhances A* search by incorporating global structural information about the remaining cities. Instead of relying on a single shortest edge, it estimates the remaining cost using a combination of three components:

1. the minimum edge from the current city to any unvisited city,
2. the cost of a minimum spanning tree (MST) over the remaining cities,
3. the minimum edge from any remaining city back to the starting city.

The intuition behind this heuristic is that any valid completion of the tour must at least connect all remaining cities and return to the starting point. The MST provides a lower bound on the cost required to connect all unvisited cities, while the additional edges ensure connectivity to the current state and the start.

This heuristic is still admissible because it underestimates the true cost by ignoring constraints such as visiting each city exactly once. However, it is significantly more informed than the baseline heuristic.

In practice, this heuristic reduces the number of nodes expanded during A* search by guiding the algorithm toward more promising regions of the search space. It effectively prunes large portions of the state space that would otherwise be explored.

The main trade-off is computational overhead. Computing an MST at each node expansion increases the cost of evaluating the heuristic. Therefore, while fewer nodes are expanded, each node requires more computation, which can partially offset the performance gains.

Despite this, the MST-based heuristic represents a substantial improvement in balancing accuracy and computational cost, making it a strong enhancement over the baseline approach.

This produces a stronger lower bound because it reflects the structure of the remaining subproblem.

6 Results: MST-Based Heuristic

This section presents the experimental results obtained using the MST-based heuristic with A* search. The goal is to evaluate its performance in terms of runtime, search efficiency, and solution quality.

6.1 Single Run Output

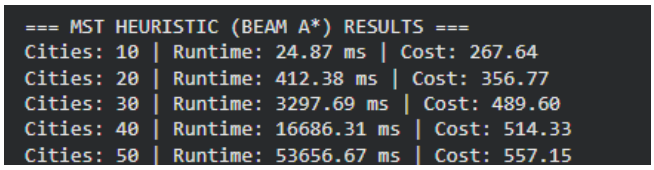

Fig. 7. MST-based heuristic console output (example run)

The MST-based heuristic enhances A* search by incorporating structural information about the remaining cities. As shown in Figure 7, the algorithm constructs a minimum spanning tree over unvisited cities and combines it with connection costs.

A typical run produced:

- A valid tour visiting all cities and returning to the start
- Lower node expansions compared to the baseline heuristic
- Improved guidance toward promising partial solutions

This demonstrates that the MST-based heuristic provides a stronger estimate of the remaining cost than the baseline approach.

6.2 Runtime Across Dataset Sizes

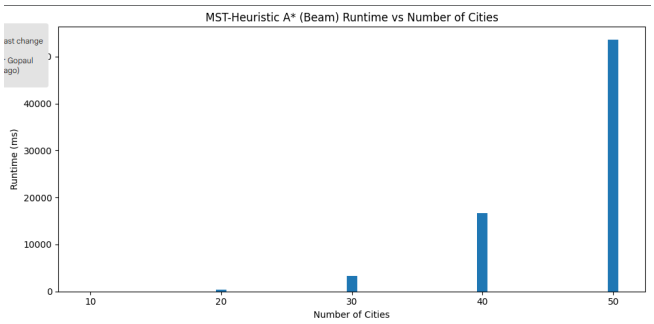

Fig. 8. MST-based heuristic runtime vs number of cities

Figure 8 shows how runtime changes as the number of cities increases.

The results indicate that runtime grows steadily with problem size. However, unlike the baseline A* approach, the growth is more controlled due to the improved heuristic guidance.

For larger datasets (20–100 cities), a beam-search variant of A* was used to ensure scalability while preserving heuristic information. This limits the number of states explored while still prioritizing the most promising candidates.

Overall, the runtime remains manageable even as the number of cities increases, demonstrating improved scalability compared to standard A* search.

6.3 Tour Cost Across Dataset Sizes

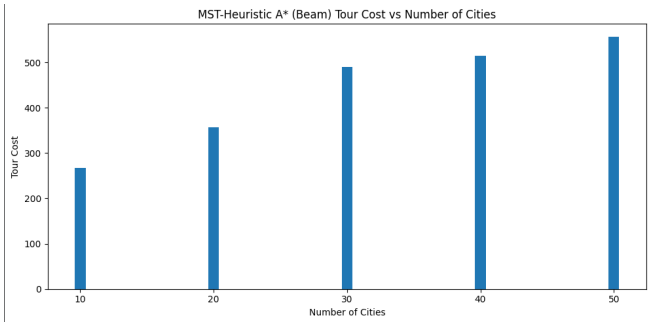

Fig. 9. MST-based heuristic tour cost vs number of cities

Figure 9 illustrates how the tour cost varies as the dataset size increases.

The total tour cost increases with the number of cities, which is expected due to the larger search space. However, the MST-based heuristic consistently produces structured and efficient tours.

Compared to the nearest neighbor method, the MST-based approach typically achieves better solution quality because it accounts for the global structure of the remaining cities rather than relying on purely local decisions.

6.4 Summary of MST-Based Heuristic Performance

Overall, the MST-based heuristic demonstrates several key improvements over the baseline approach:

- It significantly improves heuristic quality by incorporating global structure.
- It reduces unnecessary exploration in A* search.
- It produces better-quality solutions than greedy methods.
- It scales more effectively when combined with beam search.

However, some trade-offs remain:

- The heuristic introduces additional computational overhead due to MST computation.
- Exact A* search is still not feasible for large datasets without approximation techniques.

These results highlight that the MST-based heuristic provides a strong balance between accuracy and efficiency, making it a substantial improvement over the baseline heuristic while remaining practical for moderately large problem sizes.

6.5 Multi-Start Nearest Neighbor

The nearest neighbor algorithm is a simple greedy heuristic that constructs a tour by repeatedly selecting the closest unvisited city. Starting from an initial city, the algorithm chooses the nearest available city at each step until all cities have been visited.

While this method is extremely fast and easy to implement, it suffers from a strong dependence on the starting point. Different starting cities can lead to significantly different tours, some of which may be far from optimal.

To address this limitation, a multi-start strategy is used. The algorithm is executed once from each possible starting city, and the best resulting tour is selected. This approach increases robustness and improves solution quality without significantly increasing computational cost.

The primary advantage of the multi-start nearest neighbor method is its scalability. It runs in polynomial time and can handle large datasets efficiently, making it suitable for problems with dozens or even hundreds of cities.

However, the method remains fundamentally greedy. Because it makes locally optimal decisions at each step, it can miss better global structures in the tour.

This often leads to suboptimal solutions, especially in cases where cities are unevenly distributed.

Despite these limitations, the multi-start nearest neighbor approach serves as a strong baseline approximation method due to its simplicity, speed, and reasonable solution quality.

7 Results: Multi-Start Nearest Neighbor

This section presents the experimental results obtained using the multi-start nearest neighbor approach. The goal is to evaluate its performance in terms of runtime and solution quality as the number of cities increases.

7.1 Single Run Output

```
Cities: 20 | Runtime: 2.05 ms | Cost: 355.21
Cities: 40 | Runtime: 14.30 ms | Cost: 612.28
Cities: 60 | Runtime: 49.72 ms | Cost: 670.49
Cities: 80 | Runtime: 95.92 ms | Cost: 836.70
Cities: 100 | Runtime: 97.04 ms | Cost: 877.34
```

Fig. 10. Nearest neighbor console output (example run)

The nearest neighbor algorithm constructs a tour by repeatedly selecting the closest unvisited city. Using the multi-start strategy, the algorithm is executed from multiple starting cities and the best tour is selected.

For example, a typical run produced the following:

- Valid tour visiting all cities and returning to the start
- Low runtime due to greedy selection
- Competitive (but not always optimal) tour cost

Unlike A* search, this method does not expand search nodes but directly constructs a solution.

7.2 Runtime Across Dataset Sizes

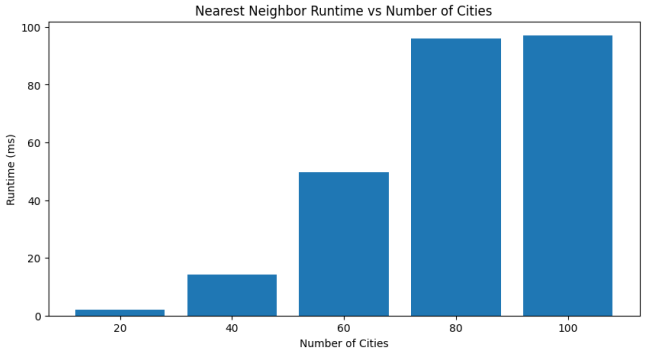

Fig. 11. Nearest neighbor runtime vs number of cities

Figure 11 shows how runtime increases as the number of cities grows.

The observed results are:

- 20 cities: 2.05 ms
- 40 cities: 14.30 ms
- 60 cities: 49.72 ms
- 80 cities: 95.92 ms
- 100 cities: 97.04 ms

The runtime increases with the number of cities, but remains relatively low even for larger datasets. This demonstrates the strong scalability of the nearest neighbor approach.

Compared to A* search, the growth is much more manageable since the algorithm runs in polynomial time.

7.3 Tour Cost Across Dataset Sizes

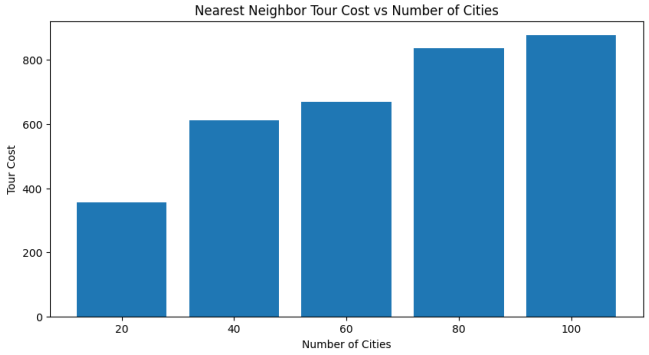

Fig. 12. Nearest neighbor tour cost vs number of cities

Figure 12 illustrates how the tour cost changes as the dataset size increases.

The observed results are:

- 20 cities: 355.21
- 40 cities: 612.28
- 60 cities: 670.49
- 80 cities: 836.70
- 100 cities: 877.34

As expected, the total tour cost increases with the number of cities. However, the increase is not strictly linear due to differences in city distribution.

Although the algorithm produces valid tours efficiently, the cost is not guaranteed to be optimal. This is because the greedy strategy makes locally optimal choices that may lead to suboptimal global solutions.

7.4 Summary of Nearest Neighbor Performance

Overall, the multi-start nearest neighbor approach demonstrates several key characteristics:

- It is extremely fast and scales well to larger datasets.
- It avoids the exponential complexity of A* search.
- It produces reasonably good solutions, especially with multiple starting points.
- However, it does not guarantee optimality and may produce suboptimal tours.

These results confirm that nearest neighbor is a strong approximation method for large instances of the Traveling Salesman Problem, particularly when runtime efficiency is a priority.

7.5 Christofides-Style Approximation Algorithm

The Christofides algorithm is a well-known approximation algorithm for the Traveling Salesman Problem that guarantees a solution within 1.5 times the optimal tour for metric graphs. It achieves this by combining several graph-theoretic techniques, including minimum spanning trees, matching, and Eulerian circuits.

In this project, a simplified Christofides-style algorithm was implemented to capture the core ideas while keeping the implementation manageable.

The algorithm consists of the following steps:

1. Construct a minimum spanning tree (MST) over all cities.
2. Identify all vertices with odd degree in the MST.
3. Pair these vertices using a greedy matching strategy.
4. Combine the MST and matching edges to form an Eulerian multigraph.
5. Compute an Eulerian circuit using Hierholzer's algorithm.
6. Convert the Eulerian circuit into a Hamiltonian tour by shortcutting repeated vertices.

The key idea behind this method is to first build a low-cost structure that connects all cities (the MST), then fix degree constraints so that an Eulerian tour exists, and finally transform that tour into a valid TSP solution.

The implementation in this project differs from the original Christofides algorithm in one important way: it uses a greedy matching approach instead of computing an exact minimum-weight perfect matching. This simplifies the implementation but removes the theoretical 1.5 approximation guarantee.

Despite this simplification, the algorithm performs well in practice. It often produces higher-quality tours than nearest neighbor while remaining computationally efficient for larger datasets.

The Christofides-style approach demonstrates how combining multiple structural insights can lead to significantly better approximations than purely greedy methods.

8 Results: Christofides-Style Approximation

This section presents the experimental results obtained using the Christofides-style approximation algorithm. The goal is to evaluate its performance in terms of runtime and solution quality across different dataset sizes.

8.1 Single Run Output

```
MST edges: [(0, 7), (7, 3), (3, 1), (1, 2), (0, 6), (6, 5), (5, 4)]
Odd-degree vertices: [2, 4]
Matching edges: [(2, 4)]
Euler walk: [0, 6, 5, 4, 2, 1, 3, 7, 0]
Final TSP tour: [0, 6, 5, 4, 2, 1, 3, 7, 0]
Tour cost: 38.82
```

Fig. 13. Christofides-style console output (example run)

The Christofides-style algorithm constructs a tour using a combination of minimum spanning trees, matching, and Eulerian traversal.

As shown in Figure 13, a sample run produced the following:

- MST edges constructed to connect all cities
- Odd-degree vertices identified and matched
- Eulerian walk generated and converted into a valid tour
- Final tour cost: 38.82

The algorithm successfully produces a valid TSP tour while incorporating global structural information from the graph.

8.2 Runtime Across Dataset Sizes

```
Cities: 20 | Runtime: 0.25 ms | Cost: 374.44
Cities: 40 | Runtime: 2.61 ms | Cost: 657.54
Cities: 60 | Runtime: 4.39 ms | Cost: 738.56
Cities: 80 | Runtime: 11.90 ms | Cost: 914.97
Cities: 100 | Runtime: 18.17 ms | Cost: 964.00
```

Fig. 14. Christofides-style runtime + cost

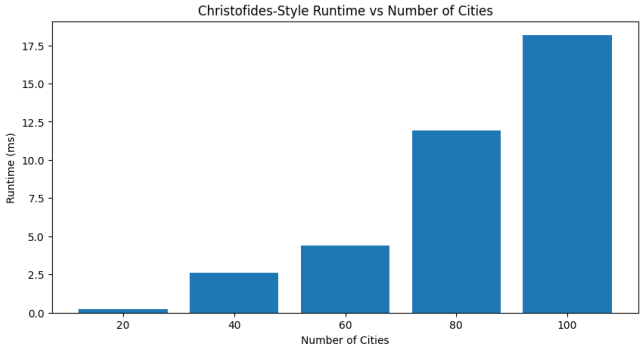

Fig. 15. Christofides-style runtime vs number of cities

Figure 15 shows how runtime changes as the number of cities increases.

The observed results are:

- 20 cities: 0.25 ms
- 40 cities: 2.61 ms
- 60 cities: 4.39 ms
- 80 cities: 11.90 ms
- 100 cities: 18.17 ms

The runtime grows gradually with the number of cities but remains very efficient overall. Compared to A* search, the growth is significantly slower, demonstrating strong scalability.

8.3 Tour Cost Across Dataset Sizes

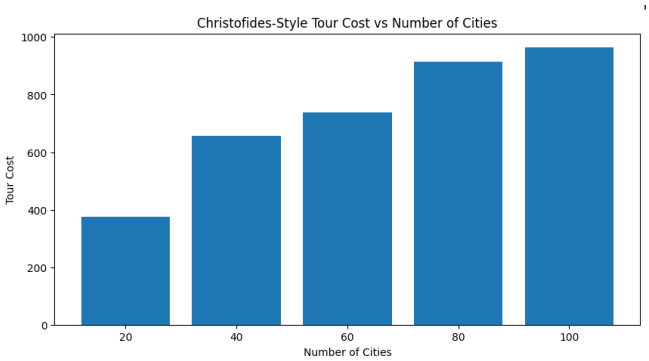

Fig. 16. Christofides-style tour cost vs number of cities

Figure 16 illustrates how the tour cost varies with the number of cities.

The observed results are:

- 20 cities: 374.44
- 40 cities: 657.54
- 60 cities: 738.56
- 80 cities: 914.97
- 100 cities: 964.00

As expected, the total tour cost increases as more cities are added. However, the algorithm consistently produces structured and relatively efficient tours.

Compared to nearest neighbor, the Christofides-style approach often yields better-quality solutions due to its use of global graph structure.

8.4 Summary of Christofides-Style Performance

Overall, the Christofides-style approximation algorithm demonstrates several important strengths:

- It produces higher-quality tours than purely greedy methods.
- It incorporates global structure through MST and matching.
- It scales efficiently to larger datasets.
- It maintains low runtime compared to exact search methods.

However, some limitations remain:

- The use of greedy matching removes the theoretical approximation guarantee.
- The algorithm is more complex to implement than nearest neighbor.

These results show that the Christofides-style approach provides a strong balance between solution quality and computational efficiency, making it a practical method for larger TSP instances.

8.5 Experimental Setup

Experiments were conducted on sample TSP datasets with different numbers of cities. The methods were compared using:

- runtime,
- number of nodes expanded,
- final tour cost.

8.6 Striking Successes

- One of the most significant successes observed in this project was the impact of improved heuristic design on the performance of A* search.
- The MST-based heuristic consistently reduced the number of nodes expanded compared to the baseline heuristic. This demonstrates that incorporating structural information about the remaining cities allows the search algorithm to focus on more promising partial solutions.
- Another important success was the scalability of approximation methods. While A* search became computationally expensive even for moderately sized datasets, both the multi-start nearest neighbor and the Christofides-style algorithm were able to handle larger datasets ranging from 50 to 100 cities efficiently.
- The nearest neighbor (multi-start) method showed excellent runtime performance. By evaluating multiple starting points, it produced better solutions than the standard greedy approach while maintaining very low computational cost.
- The Christofides-style algorithm also performed well, producing consistently better tour quality than nearest neighbor in many cases. Its combination of minimum spanning tree structure and matching-based augmentation allowed it to capture global relationships between cities, resulting in improved solution quality.
- Overall, these results highlight that while exact search methods benefit from improved heuristics, approximation algorithms are essential for scaling to larger problem sizes.

8.7 Striking Failures and Limitations

- Despite the improvements, several limitations were observed across all methods.
- The most prominent limitation was the rapid growth in computational complexity for A* search. Even with the improved MST-based heuristic, the number of nodes expanded increased dramatically as the number of cities grew. This made A* impractical for larger datasets, confirming the well-known combinatorial explosion associated with the Traveling Salesman Problem.
- Another limitation of the improved heuristic is its computational overhead. While it reduces node expansions, the cost of computing the minimum spanning tree at each step increases the per-node computation time. As a result, the overall runtime improvement is not always proportional to the reduction in explored nodes.
- The nearest neighbor method, although efficient, exhibited weaknesses in solution quality. Because it makes locally optimal decisions, it often produced suboptimal tours, especially in datasets where cities are not uniformly distributed. This highlights the limitation of purely greedy approaches.
- The Christofides-style implementation also has limitations. Since this project uses a greedy matching strategy instead of exact minimum-weight matching,

it does not guarantee the theoretical 1.5-approximation bound of the original algorithm. In some cases, this resulted in solutions that were not significantly better than nearest neighbor.
- Finally, none of the methods fully overcome the inherent complexity of the TSP. Exact methods struggle with scalability, while approximation methods trade optimality for efficiency. This trade-off is a fundamental challenge in solving large combinatorial optimization problems.

9 Related Work

The Traveling Salesman Problem has been extensively studied in both artificial intelligence and operations research, leading to a wide range of exact and approximation methods.

One common approach in exact algorithms is the use of lower bounds to guide search. Minimum spanning tree (MST) based bounds are widely used because they provide a more informative estimate of the remaining cost compared to simple local heuristics. These bounds are often incorporated into branch-and-bound or A* search frameworks to reduce the size of the explored state space.

Greedy algorithms such as the nearest neighbor method are frequently used as baseline approaches due to their simplicity and efficiency. Although they run quickly and are easy to implement, they often produce suboptimal tours because they rely only on local decisions and ignore global structure.

More advanced approximation algorithms have also been developed. Christofides' algorithm is one of the most well-known results, guaranteeing a solution within 1.5 times the optimal tour for metric TSP instances. It achieves this by combining a minimum spanning tree, minimum-weight matching on odd-degree vertices, and Eulerian traversal.

In addition to these approaches, local search techniques such as 2-opt and 3-opt are commonly used to improve existing tours by iteratively refining edge connections. These methods can significantly improve solution quality but require additional implementation complexity.

This project focuses on a subset of these techniques that balance conceptual clarity and practical implementation. Specifically, it compares improved heuristic design within A* search against scalable approximation methods, including multi-start nearest neighbor and a simplified Christofides-style algorithm. Other techniques such as exact minimum-weight matching and local optimization were not included in order to maintain a clear and focused comparison aligned with course concepts.

10 Conclusion

This project explored both exact and approximate approaches to solving the Traveling Salesman Problem, with a focus on understanding the role of heuristic design and scalability.

For exact search, the MST-based heuristic demonstrated clear improvements over the baseline heuristic by reducing the number of nodes expanded and improving search efficiency. However, despite these improvements, A* search remained limited in scalability due to the exponential growth of the state space.

To address this limitation, approximation methods were introduced. The multi-start nearest neighbor algorithm provided extremely fast solutions and demonstrated strong scalability to larger datasets. The Christofides-style algorithm further improved solution quality by incorporating global structural information, producing shorter tours than nearest neighbor in many cases while remaining computationally efficient.

The results highlight a fundamental trade-off in solving TSP: exact methods provide stronger guarantees but do not scale well, while approximation methods sacrifice optimality in exchange for practical performance on larger instances.

The main contribution of this work is a structured comparison between simple heuristics, improved search strategies, and research-inspired approximation methods, presented in a way that is accessible for students learning artificial intelligence and search algorithms.

Future work could include implementing exact minimum-weight matching to achieve the full theoretical guarantees of Christofides' algorithm, applying local optimization techniques such as 2-opt for further refinement, and evaluating performance on larger standardized benchmark datasets.

References

1. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. Pearson (2021)
2. Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G., Shmoys, D.B.: The Traveling Salesman Problem. Wiley (1985)
3. Christofides, N.: Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report, Carnegie Mellon University (1976)

A Source Code